

Лабораторная №7. Добавление поддержки интерфейса SPI

Цель работы

Изучить принципы работы SPI, настроить контроллер Allwinner D1 в Device Tree, включить необходимые драйверы ядра Linux и проверить обмен с SPI-устройством из пользовательского пространства.

Оборудование и предварительные требования

- одноплатный компьютер Lichee RV Dock с подготовленной SD-картой;
- компьютер со средой сборки ядра Linux и исходными файлами Device Tree;
- результаты лабораторных №4–6;
- USB-UART-преобразователь с логическими уровнями 3,3 В;
- перемычка для теста MOSI–MISO;
- Python 3 и модуль spidev на плате.

Опасность повреждения

Все внешние соединения с GPIO выполняйте только при выключенном питании платы. Выводы Allwinner D1 работают с логическими уровнями 3,3 В; подача 5 В может повредить плату.

Ключевые понятия

Определение

SPI (Serial Peripheral Interface) — синхронный последовательный интерфейс, в котором контроллер формирует тактовый сигнал и выбирает периферийное устройство отдельной линией CS.

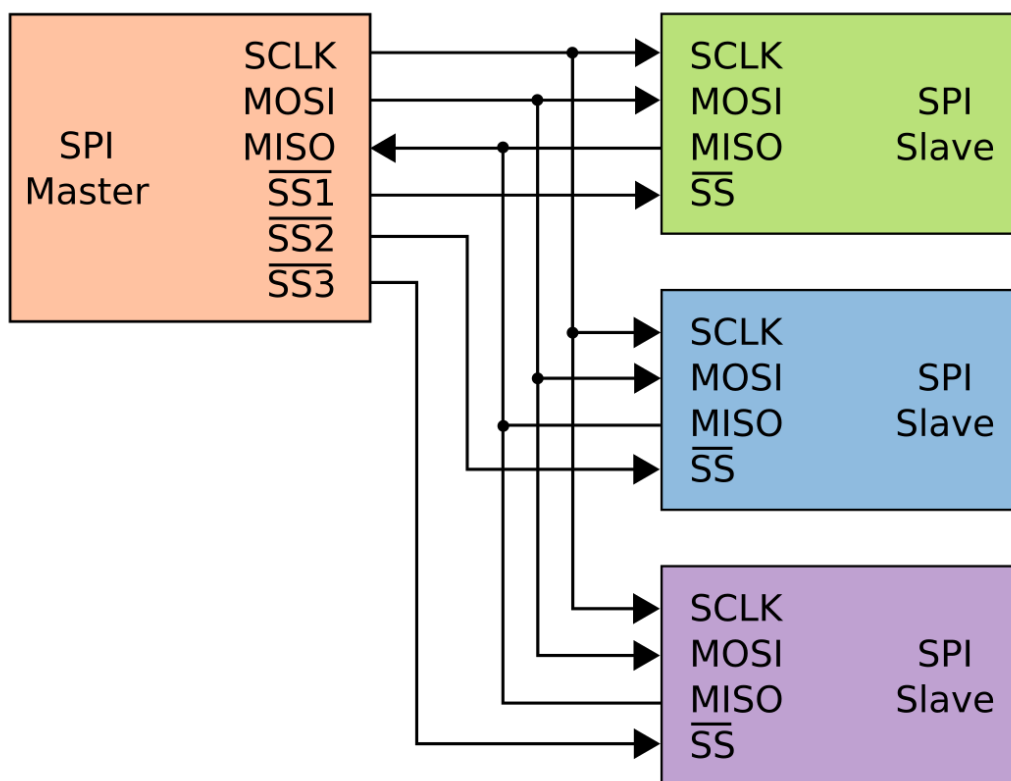


Рис. 1 – Топология SPI с общими линиями данных и отдельными сигналами выбора периферийных устройств

SPI использует четыре основных сигнала:

- SCK — тактовый сигнал;
- MOSI — данные от контроллера к периферийному устройству;
- MISO — данные от периферийного устройства к контроллеру;
- CS — выбор конкретного периферийного устройства.

Линии MOSI, MISO и SCK могут быть общими для нескольких устройств, а линия CS должна быть отдельной для каждого из них. Специализированный вывод контроллера SPI или обычный GPIO может формировать CS.

Определение

Полнодуплексный обмен — одновременная передача данных по MOSI и приём данных по MISO на каждом такте SCK.

Каждая операция SPI принимает столько же байтов, сколько передаёт, даже если принятые байты не содержат полезной информации.

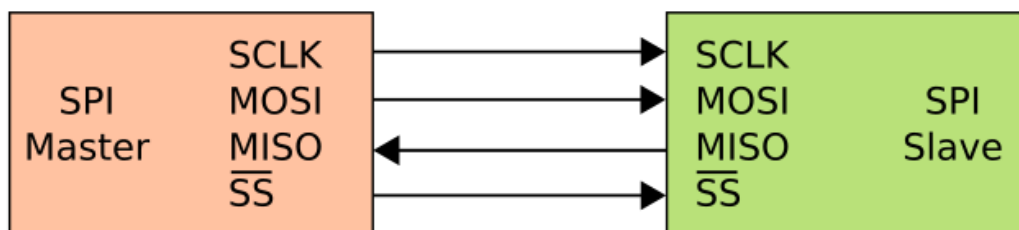


Рис. 2 – Подключение одного SPI-устройства к Lichee RV Dock

Порядок выполнения

Анализ аппаратной конфигурации

На Lichee RV Dock сигналы SPI1 выведены на контакты PD10–PD15 как альтернативные функции GPIO.

Sipeed Lichee RV DOCK pinout diagram

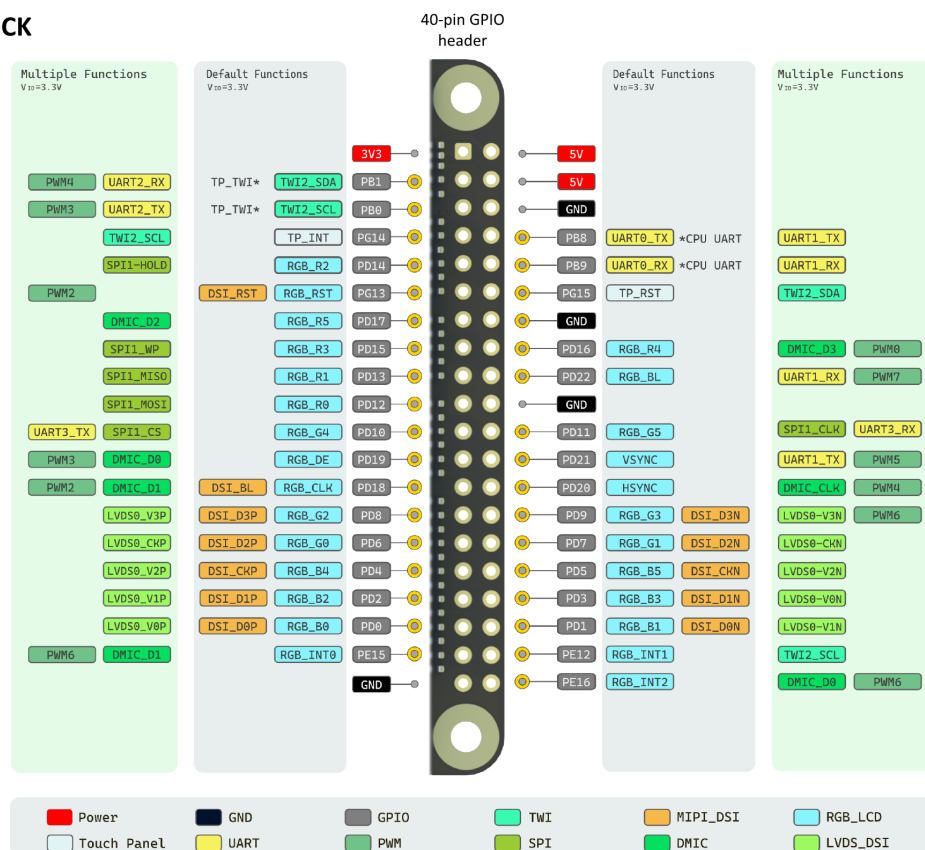


Рис. 3 – Распиновка Lichee RV Dock: сигналы SPI1 на выводах PD10–PD15

По умолчанию эти выводы могут быть назначены RGB-интерфейсу дисплея. Перед активацией SPI проверьте исходный Device Tree и исключите конфликт функций.

Внимание

Один физический вывод нельзя одновременно использовать для RGB и SPI. Если узел дисплея занимает PD10–PD15, отключите конфликтующую функцию в Device Tree до включения SPI1.

Анализ описания SPI в Device Tree

Исходные файлы платы находятся в каталоге `arch/riscv/boot/dts/allwinner/`. Файл `sun20i-d1-lichee-rv-dock.dts` включает описание базовой платы, а оно, в свою очередь, включает общие файлы `.dtsi` с контроллерами SoC.

Определение

Device Tree Source Include (DTSI) — включаемый исходный файл дерева устройств с общими описаниями аппаратных блоков и выводов.

В общем файле объявлены группы выводов двух контроллеров SPI:

```
/omit-if-no-ref/
spi0_pins: spi0-pins {
    pins = "PC2", "PC3", "PC4", "PC5";
    function = "spi0";
};

/omit-if-no-ref/
spi1_pb_pins: spi1-pb-pins {
    pins = "PB0", "PB8", "PB9", "PB10", "PB11", "PB12";
    function = "spi1";
};

/omit-if-no-ref/
spi1_pd_pins: spi1-pd-pins {
    pins = "PD10", "PD11", "PD12", "PD13", "PD14", "PD15";
    function = "spi1";
};
```

Директива `/omit-if-no-ref/` исключает неиспользуемый узел из итогового DTB. Контроллеры объявлены отдельно и изначально отключены:

```
spi0: spi@4025000 {
    compatible = "allwinner,sun20i-d1-spi",
                 "allwinner,sun50i-r329-spi";
    reg = <0x4025000 0x1000>;
    interrupts = <31 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&ccu CLK_BUS_SPI0>, <&ccu CLK_SPI0>;
    clock-names = "ahb", "mod";
    resets = <&ccu RST_BUS_SPI0>;
    dmas = <&dma 22>, <&dma 22>;
    dma-names = "rx", "tx";
    num-cs = <1>;
```

```

        status = "disabled";
        #address-cells = <1>;
        #size-cells = <0>;
    };

    spi1: spi@4026000 {
        compatible = "allwinner,sun20i-d1-spi-dbi",
            "allwinner,sun50i-r329-spi-dbi",
            "allwinner,sun50i-r329-spi";
        reg = <0x4026000 0x1000>;
        interrupts = <32 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&ccu CLK_BUS_SPI1>, <&ccu CLK_SPI1>;
        clock-names = "ahb", "mod";
        resets = <&ccu RST_BUS_SPI1>;
        dmas = <&dma 23>, <&dma 23>;
        dma-names = "rx", "tx";
        num-cs = <1>;
        status = "disabled";
        #address-cells = <1>;
        #size-cells = <0>;
    };

```

Свойство `compatible` связывает аппаратный узел с поддерживающим его драйвером. При обнаружении активного совместимого узла драйвер выполняет инициализацию устройства.

Активация SPI1 и SPIDEV

Для включения SPI1 на группе PD10–PD15 добавьте в DTS платы:

```

&spi1 {
    pinctrl-0 = <&spi1_pd_pins>;
    pinctrl-names = "default";
    status = "okay";
};

```

Для доступа из пользовательского пространства требуется драйвер SPIDEV и дочерний узел устройства. В этой работе используется совместимый идентификатор, присутствующий в таблице драйвера:

```

&spi1 {
    pinctrl-0 = <&spi1_pd_pins>;
    pinctrl-names = "default";
    status = "okay";

    spidev0: spidev@0 {
        compatible = "rohm,dh2228fv";
        reg = <0>;
        spi-max-frequency = <1000000>;
    };
};

```

Внимание

Идентификатор `rohm,dh2228fv` применяется здесь как учебный способ привязать `SPIDEV`. Для реального периферийного устройства используйте его штатный `compatible` и драйвер, если они существуют.

Настройка ядра

Используйте конфигурацию, подготовленную в лабораторной №4. Проверьте и включите параметры:

```
CONFIG_SPI=y
CONFIG_SPI_MASTER=y
CONFIG_SPI_SUN6I=y
CONFIG_SPI_SPIDEV=y
CONFIG_DMA_SUN6I=y
```

Драйвер `SPI_SUN6I` исторически назван по первому поддерживаемому семейству, но применяется и к совместимым контроллерам новых SoC, включая `sun20i`. Драйвер `DMA_SUN6I` нужен контроллеру для доступа к DMA-каналам в рассматриваемой конфигурации.

Внимание

Рабочая конфигурация требует согласованных изменений Device Tree и ядра. При отсутствии одного из нужных драйверов устройство `/dev/spidev*` может не появиться, а контроллер может завершить инициализацию с ошибкой.

Соберите ядро и DTB, обновите компоненты на SD-карте и загрузите плату согласно предыдущим лабораторным.

Проверка устройства

После загрузки проверьте наличие символьного устройства:

```
ls -l /dev/spidev*
dmesg | grep -i spi
```

Проверка

Ожидаемый результат. Для шины 1 и устройства с `reg = <0>` появляется `/dev/spidev1.0`. Фактическое имя подтвердите по выводу системы.

Полнодуплексный тест

При полностью отключённом питании соедините выводы `MOSI` и `MISO`, затем загрузите плату и запустите тест:

```
import time
```

```

import spidev

def spi_sequential_test(bus=1, device=0, speed=1_000_000):
    spi = spidev.SpiDev()
    spi.open(bus, device)
    spi.max_speed_hz = speed
    spi.mode = 0
    spi.lsbfirst = False

    try:
        print("Последовательный тест SPI запущен")
        for value in range(256):
            response = spi.xfer2([value])
            print(
                f"Отправлено: {value:3d} (0x{value:02X}) | "
                f"Получено: {response[0]:3d} (0x{response[0]:02X}) | "
                f"Двоично: {response[0]:08b}"
            )
            time.sleep(0.01)
    finally:
        spi.close()

if __name__ == "__main__":
    spi_sequential_test(bus=1, device=0, speed=1_000_000)

```

Опасность повреждения

Устанавливайте и снимайте перемычку MOSI-MISO только при выключенном питании. Не соединяйте сигнальные выводы с питанием или землёй и не подавайте на них 5 В.

Проверка

Ожидаемый результат. При исправной перемычке каждый принятый байт совпадает с переданным. Несовпадение требует проверки имени `/dev/spidev*`, распиновки, режима и соединения MOSI-MISO.

```
[root@localhost my_spi_device]# python3 spi.py
SPI последовательный тест запущен...
Отправляю данные: 0, 1, 2, 3, ...
-----
Отправлено:  0 (0x00) | Получено:  0 (0x00) | Бинарно: 00000000
Отправлено:  1 (0x01) | Получено:  1 (0x01) | Бинарно: 00000001
Отправлено:  2 (0x02) | Получено:  2 (0x02) | Бинарно: 00000010
Отправлено:  3 (0x03) | Получено:  3 (0x03) | Бинарно: 00000011
Отправлено:  4 (0x04) | Получено:  4 (0x04) | Бинарно: 00000100
Отправлено:  5 (0x05) | Получено:  5 (0x05) | Бинарно: 00000101
Отправлено:  6 (0x06) | Получено:  6 (0x06) | Бинарно: 00000110
Отправлено:  7 (0x07) | Получено:  7 (0x07) | Бинарно: 00000111
Отправлено:  8 (0x08) | Получено:  8 (0x08) | Бинарно: 00001000
Отправлено:  9 (0x09) | Получено:  9 (0x09) | Бинарно: 00001001
Отправлено: 10 (0x0A) | Получено: 10 (0x0A) | Бинарно: 00001010
```

Рис. 4 – Результат полнодуплексного теста SPI с переключкой между MOSI и MISO

Задание

1. Проанализируйте Device Tree платы Lichee RV Dock и найдите описание SPI1 и группы `spi1_pd_pins`.
2. Убедитесь, что выводы PD10–PD15 не заняты конфликтующим узлом.
3. Активируйте SPI1 и добавьте дочерний узел для доступа через SPIDEV.
4. Включите `CONFIG_SPI`, `CONFIG_SPI_MASTER`, `CONFIG_SPI_SUN6I`, `CONFIG_SPI_SPIDEV` и `CONFIG_DMA_SUN6I`.
5. Соберите ядро и DTB, обновите загрузочную карту и загрузите систему.
6. Подтвердите наличие `/dev/spidev*`.
7. Проведите тест с переключкой MOSI-MISO и зафиксируйте результат.
8. Продемонстрируйте работу преподавателю.

Контрольные вопросы

1. Какие четыре сигнала используются в SPI и за что отвечает каждый?
2. Чем топология SPI отличается от шинной топологии I2C?
3. Почему узел SPI в Device Tree по умолчанию имеет `status = "disabled"`?
4. Зачем нужен SPIDEV и какое устройство появляется в `/dev` после его активации?
5. Какой идентификатор `compatible` используется в этой работе для привязки SPIDEV?
6. Почему для рассматриваемой конфигурации Allwinner D1 требуется

CONFIG_DMA_SUN6I?

Требования к отчёту

Отчёт должен содержать:

- цель работы;
- фрагменты Device Tree до и после изменения;
- включённые параметры конфигурации ядра;
- вывод проверки /dev/spidev* и значимые сообщения dmesg;
- схему или фотографию перемычки MOSI-MISO;
- результат полнодуплексного теста;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.